

EXPERIMENT 3

Sanket Patil D20A-25

Aim

To create a Cryptocurrency using Python and perform mining in Blockchain.

Objectives

The objectives of this experiment are:

1. To understand the working of cryptocurrency and blockchain.
2. To implement a blockchain using Python.
3. To create a decentralized Peer-to-Peer (P2P) blockchain network.
4. To perform cryptocurrency transactions.
5. To perform Proof-of-Work mining.
6. To implement the longest-chain consensus mechanism.
7. To remove pending transactions after they are added to a block.

Theory

1. Blockchain

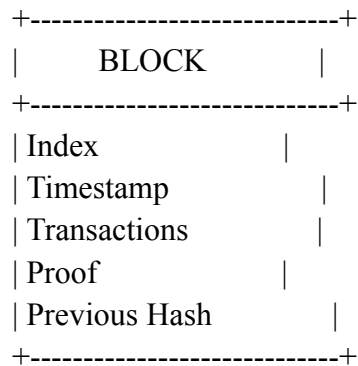
A **blockchain** is a distributed digital ledger that stores data in a sequence of blocks.

Each block contains information such as:

- Index
- Timestamp
- Transactions
- Proof
- Previous Hash

Each block is connected to the previous block through its hash. Therefore, if the contents of an earlier block are changed, its hash changes and the subsequent blocks become invalid.

Structure of a Block



2. Cryptocurrency

A **cryptocurrency** is a digital currency that uses cryptographic techniques and blockchain technology to record transactions without requiring a central authority.

In this experiment, a simple cryptocurrency called **HadCoin** is implemented using Python.

A HadCoin transaction contains:

Sender
Receiver
Amount

Example:

```
{
  "sender": "Sanket Patil",
  "receiver": "Alice",
  "amount": 50
}
```

The source experiment similarly represents transactions using sender, receiver and amount.

3. Transactions

When a transaction is submitted, it is not immediately stored inside a block.

Instead, it is first stored in the node's pending transaction list:

self.transactions

When a miner successfully mines a new block, the pending transactions are included in that block.

For example:

Sanket → Alice
50 HadCoin

Initially:

Pending Transactions

↓

[Transaction]

After mining:

Block 2

└─ Transaction

4. Mempool

A **mempool**, or memory pool, is a temporary pool containing valid transactions received by a blockchain node that have not yet been included in a block.

In this experiment:

self.transactions

acts as a simplified mempool.

After the transactions are included in a block, the pending list is cleared:

self.transactions = []

This prevents the same transactions from being repeatedly included in later blocks.

5. Mining

Mining is the process of creating a new block by solving a computational Proof-of-Work problem.

The program calculates:

```
hashlib.sha256(  
    str(new_proof**2 - previous_proof**2).encode()  
)hexdigest()
```

The mining process continues until the generated SHA-256 hash starts with:

0000

The proof satisfying this condition is then used to create the new block.

Mining Process

Previous Proof



Generate New Proof



Calculate SHA-256



Does hash start with 0000?



NO —————→ Try another proof



YES



Create New Block

6. Proof-of-Work

Proof-of-Work is a consensus mechanism that requires computational effort to find a valid proof.

In this experiment, the condition is:

SHA-256 hash starts with "0000"

For example:

Proof = 500

Hash = a82f...

✗ Invalid

Proof = 501

Hash = 91bc...

✗ Invalid

Proof = 502

Hash = 0000ab...

✓ Valid

The actual proof value is generated dynamically and may be different on every execution.

7. Mining Reward

A miner receives a reward after successfully mining a block.

In this implementation, the mining function adds a transaction:

```
blockchain.add_transaction(  
    sender=node_address,  
    receiver='Richard',  
    amount=1  
)
```

Therefore, the miner receives a reward of **1 HadCoin**, represented as a transaction in the mined block.

Conceptually:

Miner

↓

Successfully mines block

↓

Receives 1 HadCoin

8. Multi-Node Blockchain

A decentralized blockchain network consists of multiple independent nodes.

In this experiment, three blockchain nodes are created:

Node	Port
------	------

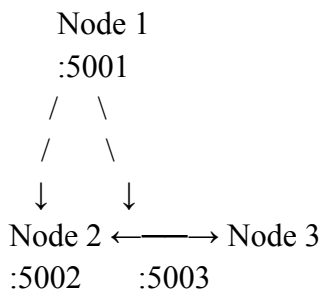
Node 1	5001
--------	------

Node 2	5002
--------	------

Node 3	5003
--------	------

Each node maintains its own copy of the blockchain and communicates with other nodes using HTTP requests.

The architecture is:



9. Peer-to-Peer Network

A **Peer-to-Peer (P2P) network** allows blockchain nodes to communicate directly with one another without depending on a central server.

Each node:

- Maintains its own blockchain.
- Maintains pending transactions.
- Can communicate with other nodes.
- Can receive blockchain information from other nodes.
- Can synchronize with the network.

The experiment uses HTTP requests between the three Flask nodes.

10. Connecting Nodes

Nodes are connected using a `connect_node()` API.

For example, Node 1 can be connected to:

`http://127.0.0.1:5002`

`http://127.0.0.1:5003`

Similarly, other nodes can be connected to their peers.

The supplied experiment demonstrates connecting the nodes using POST requests to `/connect_node`. The screenshots on page 7 show Node 1 and Node 2 being connected to their peers.

11. Longest-Chain Consensus

Different blockchain nodes may temporarily have different versions of the blockchain.

For example:

Node 1 → 3 blocks

Node 2 → 5 blocks

Node 3 → 5 blocks

The network needs a mechanism to determine which blockchain should be accepted.

This experiment uses the **Longest-Chain Rule**.

The `replace_chain()` function:

1. Contacts connected nodes.
2. Obtains their chains.
3. Checks whether the received chains are valid.
4. Compares their lengths.
5. Replaces the current chain if a longer valid chain is found.

Example

Node 1

Blockchain length = 2

Node 2

Blockchain length = 4

Node 3

Blockchain length = 4

Node 1 receives the chains and discovers that Node 2 has a longer valid chain.

Therefore:

Node 1

↓

replace_chain()

↓

Adopts Node 2's chain

Eventually the nodes converge toward the same longest valid chain.

12. Procedure

Step 1 — Prepare the Lab Files

The original experiment uses the following files:

hadcoin.py

hadcoin_node_5001.py

hadcoin_node_5002.py

hadcoin_node_5003.py

nodes.json

transaction.json

These files are listed in the supplied experiment.

For the DColab implementation, these components can instead be created directly as Python classes/functions in the notebook.

Step 2 — Check Python

Check the installed Python version:

```
!python --version
```

The reference experiment used Python 3.14.7.

Step 3 — Install Required Libraries

Install Flask and Requests:

```
!pip install Flask requests
```

The supplied experiment uses Flask for the node APIs and Requests for HTTP communication between nodes.

Step 4 — Start Three Blockchain Nodes

The experiment creates three nodes:

Node 1 → 5001

Node 2 → 5002

Node 3 → 5003

The original experiment runs them separately using:

```
.\env\Scripts\python.exe .\hadcoin_node_5001.py
```

```
.\env\Scripts\python.exe .\hadcoin_node_5002.py
```

```
.\env\Scripts\python.exe .\hadcoin_node_5003.py
```

The three nodes run simultaneously.

Step 5 — Fetch Initial Blockchain

Using Postman:

GET http://127.0.0.1:5001/get_chain

Initially, the node contains only the Genesis Block:

Index = 1

Previous Hash = 0

Proof = 1

Transactions = []

The same initial-chain test is shown in the supplied experiment on page 5.

Repeat for:

GET `http://127.0.0.1:5002/get_chain`

and:

GET `http://127.0.0.1:5003/get_chain`

Step 6 — Connect the Nodes

Use:

POST `http://127.0.0.1:5001/connect_node`

with JSON:

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5003"
  ]
}
```

Similarly, connect the other nodes.

The expected response indicates that the nodes have been connected. The supplied Postman screenshots demonstrate this operation.

Step 7 — Add a Cryptocurrency Transaction

Send:

POST `http://127.0.0.1:5001/add_transaction`

with:

```
{
  "sender": "Sanket Patil",
  "receiver": "Alice",
}
```

```
"amount": 50
}
```

Expected response:

```
{
  "message": "This transaction will be added to Block 2"
}
```

The reference experiment demonstrates the same `/add_transaction` operation with a transaction from the student to Alice for 50 units.

Step 8 — Mine a Block

Use:

GET `http://127.0.0.1:5001/mine_block`

The node performs Proof-of-Work.

It finds a proof satisfying:

SHA-256 hash starts with 0000

The block contains:

Transaction

+

Mining Reward

The miner receives:

1 HadCoin

The reference output on page 9 shows the transaction of 50 HadCoin and the mining reward of 1 HadCoin in the mined block.

Step 9 — Replace Chain Using Consensus

Now use:

GET http://127.0.0.1:5002/replace_chain

The node compares its blockchain with those of connected peers.

If another node has a longer valid blockchain, it replaces its current chain.

The response indicates that the chain has been replaced by the longest valid chain when appropriate.

Step 10 — Fetch Updated Blockchain

Use:

GET http://127.0.0.1:5002/get_chain

The node should now show the updated blockchain.

The supplied experiment demonstrates retrieving the updated chain after consensus.

Step 11 — Validate Blockchain

Use:

GET http://127.0.0.1:5001/is_valid

Expected response:

```
{  
  "message": "All good. The Blockchain is valid."  
}
```

The supplied experiment demonstrates this successful validation

Step 12 Verify Pending Transactions Are Removed

After mining, the pending transaction list should be empty.

Use:

GET http://127.0.0.1:5001/get_transactions

Expected:

[]

The supplied experiment specifically verifies that transactions are removed after being included in a block.

The implementation uses:

```
self.transactions = []
```

after the transactions are assigned to the block.

Result / Observation

✓ Create Three Blockchain Nodes

We will create three independent `Blockchain` objects, each representing a node in our Peer-to-Peer network. Each node will operate on a different port and maintain its own blockchain, pending transactions, and a list of connected peer nodes.

```
[4]
✓ Os
# Instantiate the three Blockchain nodes
# We'll assign a unique 'node_address' for each node for mining rewards

# Node 1 (runs on port 5001)
node1_blockchain = Blockchain()
node1_address = "http://127.0.0.1:5001"

# Node 2 (runs on port 5002)
node2_blockchain = Blockchain()
node2_address = "http://127.0.0.1:5002"

# Node 3 (runs on port 5003)
node3_blockchain = Blockchain()
node3_address = "http://127.0.0.1:5003"

print("Three HadCoin blockchain nodes initialized.")
print(f"Node 1 Address: {node1_address}")
print(f"Node 2 Address: {node2_address}")
print(f"Node 3 Address: {node3_address}")
```

```
✓
Three HadCoin blockchain nodes initialized.
Node 1 Address: http://127.0.0.1:5001
Node 2 Address: http://127.0.0.1:5002
Node 3 Address: http://127.0.0.1:5003
```

```

❶ # Start Node 3 in a daemon thread
thread3 = threading.Thread(target=run_node3, daemon=True)
thread3.start()

# Give some time for the servers to start up
time.sleep(3)

print("Node 1 running at http://127.0.0.1:5001")
print("Node 2 running at http://127.0.0.1:5002")
print("Node 3 running at http://127.0.0.1:5003")
print("Servers started in daemon threads.")

... * Serving Flask app '__main__'
* Serving Flask app '__main__'
* Debug mode: off
* Serving Flask app '__main__'
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
* Debug mode: off
* Debug mode: off
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:Press CTRL+C to quit
Node 1 running at http://127.0.0.1:5001
Node 2 running at http://127.0.0.1:5002
Node 3 running at http://127.0.0.1:5003
Servers started in daemon threads.

```

```

INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:19:18] "GET /get_chain HTTP/1.1" 200 -
... INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:19:18] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:19:18] "GET /get_chain HTTP/1.1" 200 -
Fetching initial blockchain for Node 1...
Node 1 Chain: {
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-30 13:17:44.481365",
      "transactions": []
    }
  ],
  "length": 1
}

Fetching initial blockchain for Node 2...
Node 2 Chain: {
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-30 13:17:44.481460",
      "transactions": []
    }
  ],
  "length": 1
}

Fetching initial blockchain for Node 3...
Node 3 Chain: {
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      ..

```

```

... INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:05] "POST /connect_node HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:05] "POST /connect_node HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:05] "POST /connect_node HTTP/1.1" 201 -
Connecting Node 1 to Node 2 and Node 3...
Successfully connected nodes to http://127.0.0.1:5001. Response: All the nodes are now connected. The HadCoin blockchain now contains the following nodes:

Connecting Node 2 to Node 1 and Node 3...
Successfully connected nodes to http://127.0.0.1:5002. Response: All the nodes are now connected. The HadCoin blockchain now contains the following nodes:

Connecting Node 3 to Node 1 and Node 2...
Successfully connected nodes to http://127.0.0.1:5003. Response: All the nodes are now connected. The HadCoin blockchain now contains the following nodes:

```

```

... INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:20] "POST /add_transaction HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:20] "POST /add_transaction HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:20] "GET /get_transactions HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:20] "GET /get_transactions HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Aug/2026 13:20:20] "GET /get_transactions HTTP/1.1" 200 -

```

Adding a transaction via Node 1...

Node 1 Response: This transaction will be added to Block 2

Adding another transaction via Node 2...

Node 2 Response: This transaction will be added to Block 2

Adding a third transaction via Node 3...

Node 3 Response: This transaction will be added to Block 2

Fetching pending transactions from Node 1 to verify...

```

Node 1 Pending Transactions: {
  "pending_transactions": [
    {
      "amount": 10,
      "receiver": "Alice",
      "sender": "Richard"
    }
  ]
}

```

Fetching pending transactions from Node 2 to verify...

```

Node 2 Pending Transactions: {
  "pending_transactions": [
    {
      "amount": 3,
      "receiver": "Bob",
      "sender": "Alice"
    }
  ]
}

```

Fetching pending transactions from Node 3 to verify...

```
0] Os print("\nMining a block on Node 3...")
response = requests.get(f'{node3_address}/mine_block')
print("Node 3 Mining Response:", json.dumps(response.json(), indent=2))

...
    "receiver": "Alice",
    "sender": "Richard"
  },
  {
    "amount": 1,
    "receiver": "Richard",
    "sender": "http://127.0.0.1:5001"
  }
]

Mining a block on Node 2...
Mining started...
Searching for valid proof...
Proof found: 533
Node 2 Mining Response: {
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "0117c13f1cb6682e9fda2d51c2475dffb40caedfd8daa56a118829738c652e54",
  "proof": 533,
  "timestamp": "2026-08-30 13:20:34.004143",
  "transactions": [
    {
      "amount": 3,
      "receiver": "Bob",
      "sender": "Alice"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "http://127.0.0.1:5002"
    }
  ]
}
```

The experiment successfully demonstrated:

- Creation of a simple cryptocurrency called HadCoin.
- Creation of a blockchain using Python and Flask.
- Creation of a three-node Peer-to-Peer blockchain network.
- Cryptocurrency transaction processing.
- Pending transaction management through a simplified mempool.
- Proof-of-Work mining.
- A mining reward of 1 HadCoin.
- Longest-chain consensus using `replace_chain()`.
- Synchronization of blockchain nodes.
- Removal of pending transactions after mining.

Conclusion

The experiment successfully implemented a simple cryptocurrency and blockchain using Python and Flask. A three-node P2P blockchain network was created using ports 5001, 5002 and 5003. Transactions were submitted and stored as pending transactions before mining.

Proof-of-Work was used to mine new blocks, and the miner received a reward of 1 HadCoin. The longest-chain consensus mechanism was implemented using `replace_chain()` to synchronize

nodes with the longest valid blockchain. Pending transactions were cleared after being included in a block, preventing them from being added repeatedly.

Thus, the experiment provided practical understanding of **blockchain, cryptocurrency transactions, mining, Proof-of-Work, mempool, P2P communication and consensus mechanisms**.